

Министерство образования и науки Российской Федерации
Пермский институт (филиал)
федеральное государство бюджетное образовательное
учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Летняя практика, 2 курс

По дисциплинам «Основы проектирования баз данных»,
«Поддержка и тестирование программных модулей»

Преподаватель: Берестов Д.Б
Выполнил работу: Каракулов Н.В
Группы: ИПс-21

2026

Пермь

Содержание

Введение	3
1. Создание проекта библиотеки классов.....	4
2. Тестирование библиотеки классов	7
3. Создание приложения WPF	9
4. Тестирование завершенного WPF приложения	18
Вывод.....	27

Введение

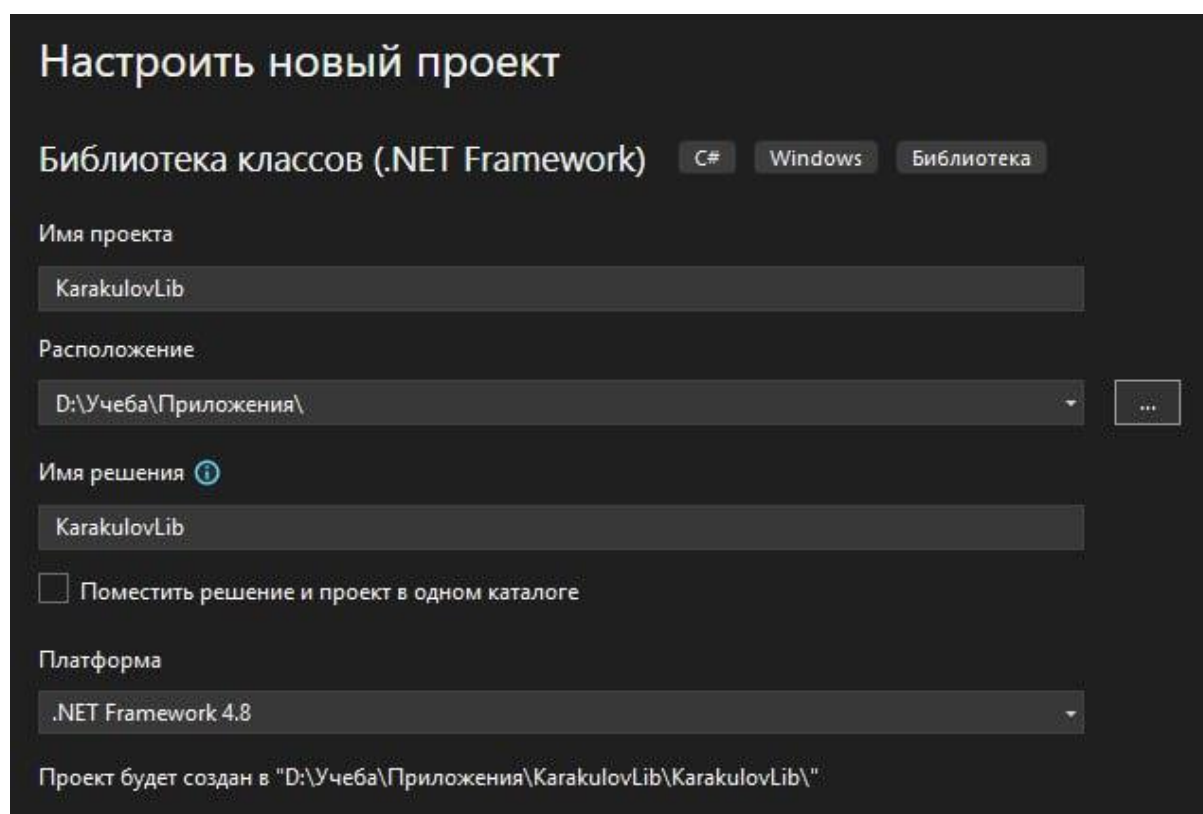
В рамках данной работы требуется разработать приложение в среде MS Visual Studio с использованием технологии WPF. Программа должна хранить данные в оперативной памяти в виде коллекции. Для взаимодействия с пользователем необходимо реализовать графический интерфейс на основе WPF, поддерживающий базовые операции с данными: добавление, редактирование, удаление, фильтрацию и поиск информации.

Основная бизнес-логика приложения должна быть вынесена в отдельную библиотеку классов .NET Framework, что обеспечит четкое разделение между логикой и пользовательским интерфейсом, а также упростит процесс тестирования. Для классов в библиотеке необходимо разработать модульные тесты, чтобы проверить их корректность и надежность.

В итоге будет создано WPF-приложение с интуитивно понятным интерфейсом, демонстрирующее работу с коллекцией данных и выполнение основных операций над ними. Кроме того, проект будет включать модульные тесты, что облегчит дальнейшую поддержку и развитие кода.

1. Создание проекта библиотеки классов

Для начала работы, необходимо создать проект в Visual Studio под названием **KarakulovLib**. Этот проект предназначен для работы с данными о задачах и их статусах. В основе библиотеки лежит контекст приложения, классы сущностей и интерфейс для управления данными (см.Рис 1).



(Рис.1 – Создание проекта в VS 2022)

В корне проекта «**KarakulovLib**» был создан класс **TaskContext**, отвечающий за хранение списка задач. Он использует коллекцию **ObservableCollection<Task>**, что обеспечивает автоматическое обновление интерфейса при изменении данных. Также в классе определено статическое свойство **TaskID**, необходимое для автоматической генерации уникальных идентификаторов.

Далее был реализован класс **Task**, представляющий задачу. Он включает следующие свойства:

Id – уникальный идентификатор задачи.

Title – название задачи.

Description – описание задачи.

Priority – приоритет задачи.

DueDate – срок выполнения (может быть *null*).

Status – текущий статус задачи.

CreatedAt – дата и время создания.

Класс **Task** содержит следующие методы:

1. **Конструктор по умолчанию** – автоматически устанавливает **CreatedAt** в текущую дату и время.
2. **Конструктор с параметрами** – позволяет инициализировать основные свойства при создании объекта.
3. **Clone()** – создает и возвращает глубокую копию задачи.
4. **CopyFrom()** – копирует все свойства (кроме **Id** и **CreatedAt**) из другой задачи.

Для стандартизации управления задачами был разработан интерфейс **ITaskManager**, определяющий базовые операции.

Основная логика работы с задачами реализована в классе **TaskManager**, который:

- 1) Реализует **ITaskManager** и **IDisposable** (для корректного освобождения ресурсов).
- 2) Предоставляет методы:

AddTask(Task task) – добавляет задачу с автоматическим назначением ID.

UpdateTask(Task task) – изменяет существующую задачу.

RemoveTask(Task task) – удаляет задачу из списка.

GetTaskById(int id) – возвращает задачу по ID.

GetAllTasks() – предоставляет перечисление всех задач.

SaveToFile(string filePath) – сохраняет список в JSON-файл.

CreateFile(string filePath) – создает новый файл с пустым списком.

LoadFromFile(string filePath) – загружает задачи из JSON-файла.

Dispose() – освобождает ресурсы.

Дополнительно класс **TaskManager** включает:

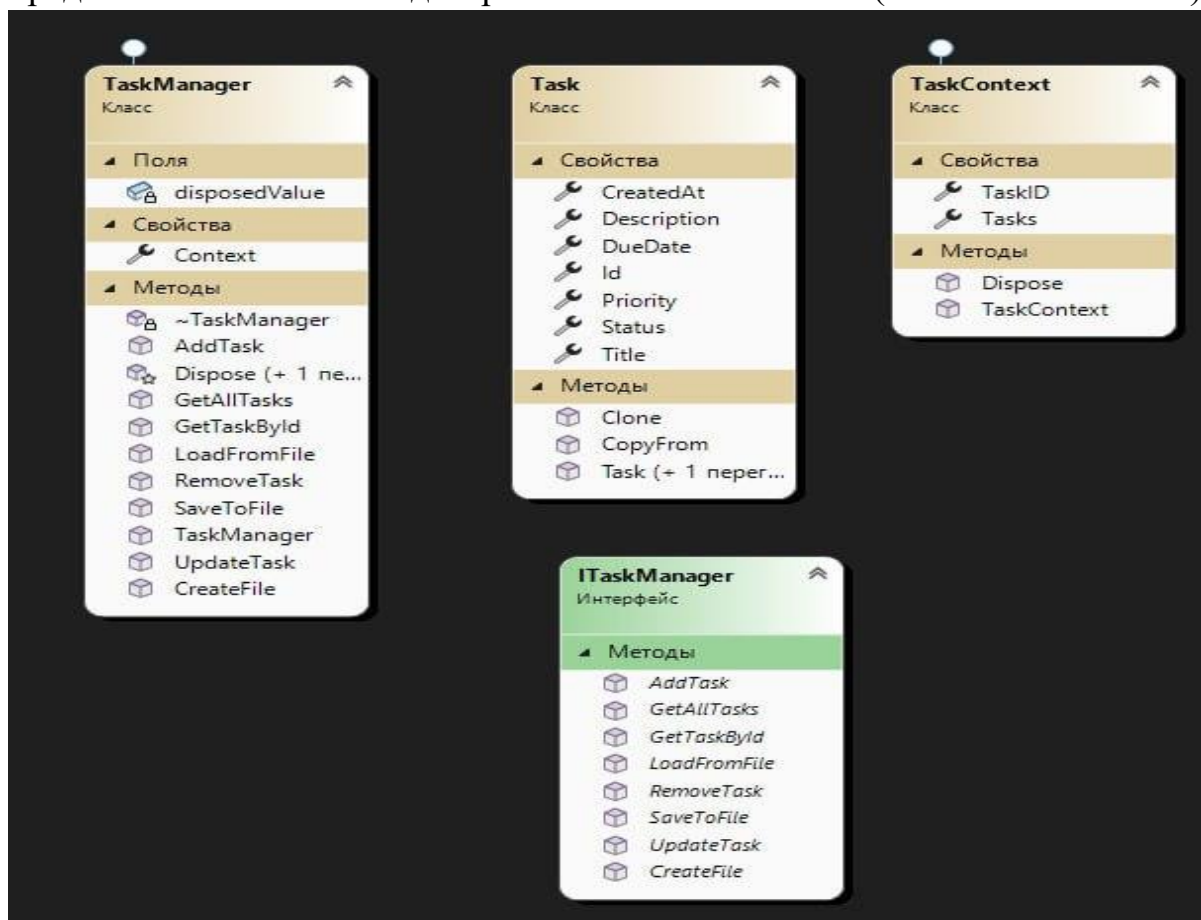
Свойство **Context** для доступа к коллекции задач.

Реализацию паттерна **Disposable**.

Обработку исключений при работе с файлами.

Автоматическое управление идентификаторами (**TaskID**).

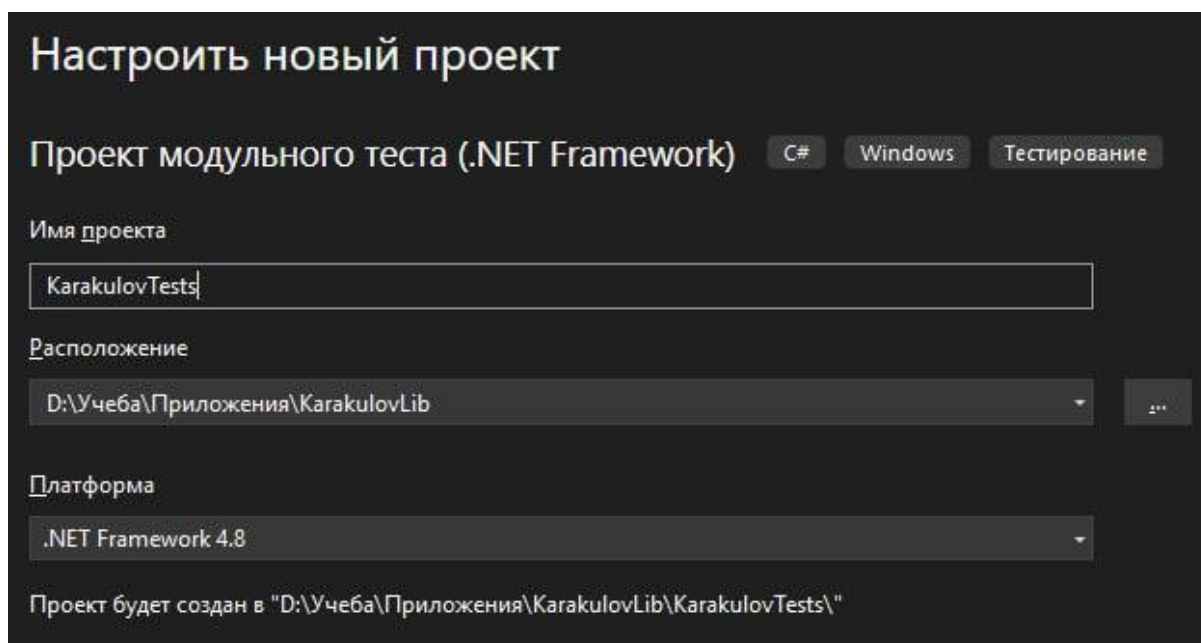
Таким образом, библиотека «**KarakulovLib**» предоставляет функционал для хранения, управления и сериализации задач, а также удобный и понятный интерфейс для взаимодействия с данными. Результат представлен на диаграмме классов (см. Рис. 2).



(Рис.2 – Диаграмма классов)

2. Тестирование библиотеки классов

После создания и реализации библиотеки классов, необходимо провести тестирование получившихся методов, для этого нужно создать проект «KarakulovTests» (См. Рис 3).



(Рис.3 – Создание проекта модульных тестов в VS 2022)

Для обеспечения изолированности тестов перед каждым тестовым случаем создается новый экземпляр TaskManager, а после выполнения - освобождаются все ресурсы. Дополнительно перед запуском каждого теста удаляется временный файл test_tasks.json (если он существует), что исключает влияние предыдущих тестовых прогонов.

Набор тестов для проверки базового функционала

1. AddTask_ShouldAddTaskToCollection

Проверка корректности добавления задач:

- 1) Создается тестовая задача
- 2) Задача добавляется в менеджер
- 3) Проверяется:

Общее количество задач (должно стать 1)

Соответствие названия добавленной задачи ожидаемому значению

2. **FindTask_ShouldFindTask**

Тестирование поиска по идентификатору:

- 1) Создается и добавляется тестовая задача
- 2) Осуществляется поиск задачи по ID
- 3) Проверяется корректность названия найденной задачи

3. **GetAllTasks_ShouldGetAllTasks**

Проверка получения полного списка:

- 1) Добавляются две тестовые задачи
- 2) Проверяется:

Общее количество задач (должно быть 2)

Корректность названия первой задачи в списке

4. **UpdateTask_ShouldUpdateExistingTask**

Тестирование обновления данных:

- 1) Создается и добавляется задача
- 2) Модифицируются её название и описание
- 3) Проверяется соответствие обновленных данных новым значениям

5. **RemoveTask_ShouldRemoveTaskFromCollection**

Проверка удаления задач:

- 1) Создается и добавляется задача (подтверждается добавление)
- 2) Задача удаляется из коллекции
- 3) Проверяется отсутствие задач в менеджере

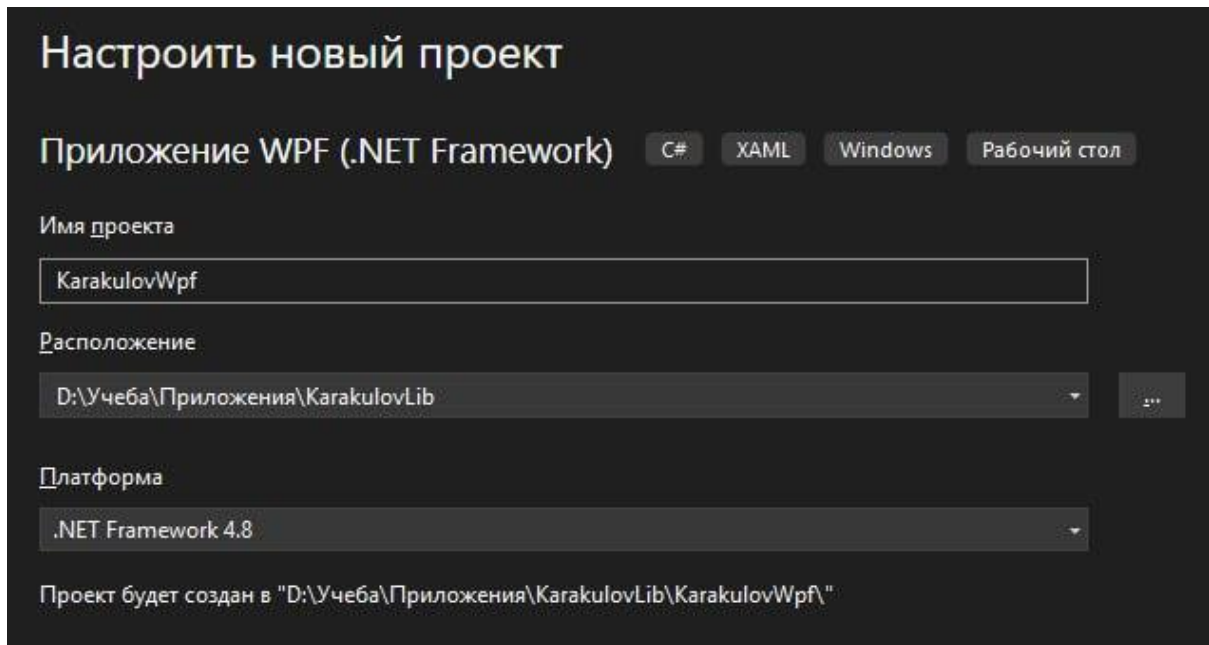
6. **SaveAndLoadFromFile_ShouldPersistTasks**

Тестирование работы с файловой системой:

- 1) Создаются и добавляются две тестовые задачи
- 2) Данные сохраняются в test_tasks.json
- 3) Создается новый экземпляр менеджера
- 4) Проверяется корректность загрузки ранее сохраненных задач

3. Создание приложения WPF

Теперь необходимо создать и правильно реализовать WPF-приложение, которое будет использовать нашу библиотеку классов для дальнейшей работы программы. Назовём его «**KarakulovWpf**» (См.Рис 4).



(Рис.4 – Создание проекта WPF (.NET Framework))

Приложение состоит из двух основных окон: главного (MainWindow) и командного (CommandWindow). Главное окно размером 900×600 пикселей с центральным расположением содержит следующие элементы:

В верхней части расположено главное меню с тремя разделами: 1) "Файл" для операций с файлами (создание/открытие/сохранение, экспорт, выход), 2) "Задачи" для CRUD-операций (добавление/редактирование/удаление, дублируется в контекстном меню), 3) "Фильтры" (по статусу - 6 вариантов, по приоритету - 5 уровней, сброс фильтров).

Под меню находится панель инструментов с полем поиска по названию задач, кнопками "Найти"/"Сбросить" и индикатором статуса поиска.

Центральную часть занимает таблица задач с 6 колонками: ID, Название, Статус, Приоритет, Срок выполнения, Дата создания - все колонки с

центрированным текстом. Внизу расположен статус-бар, отображающий количество загруженных задач.

Окно обеспечивает полный цикл работы с задачами через меню и контекстные действия, поддерживает фильтрацию и поиск, имеет строгую структуру и единообразное оформление элементов.

Окно «CommandWindow»:

Окно команд служит для работы с задачами - создания новых и редактирования существующих. Появляется при необходимости добавить новую задачу или изменить параметры существующей.

Функционал окна:

- 1) Создание новых задач
- 2) Редактирование имеющихся задач
- 3) Ввод/корректировка всех параметров задачи

Элементы управления:

1. Поля ввода:

- 1) "Название" (TitleTextBox) - обязательное поле для краткого наименования
- 2) "Описание" (DescriptionTextBox) - многострочное поле с прокруткой
- 3) "Приоритет" (PriorityComboBox) - выпадающий список важности
- 4) "Срок выполнения" (DueDatePicker) - выбор даты
- 5) "Статус" (StatusComboBox) - текущее состояние задачи

2. Управляющие кнопки:

- 1) "ОК" - сохранение данных
- 2) "Отмена" - закрытие без сохранения

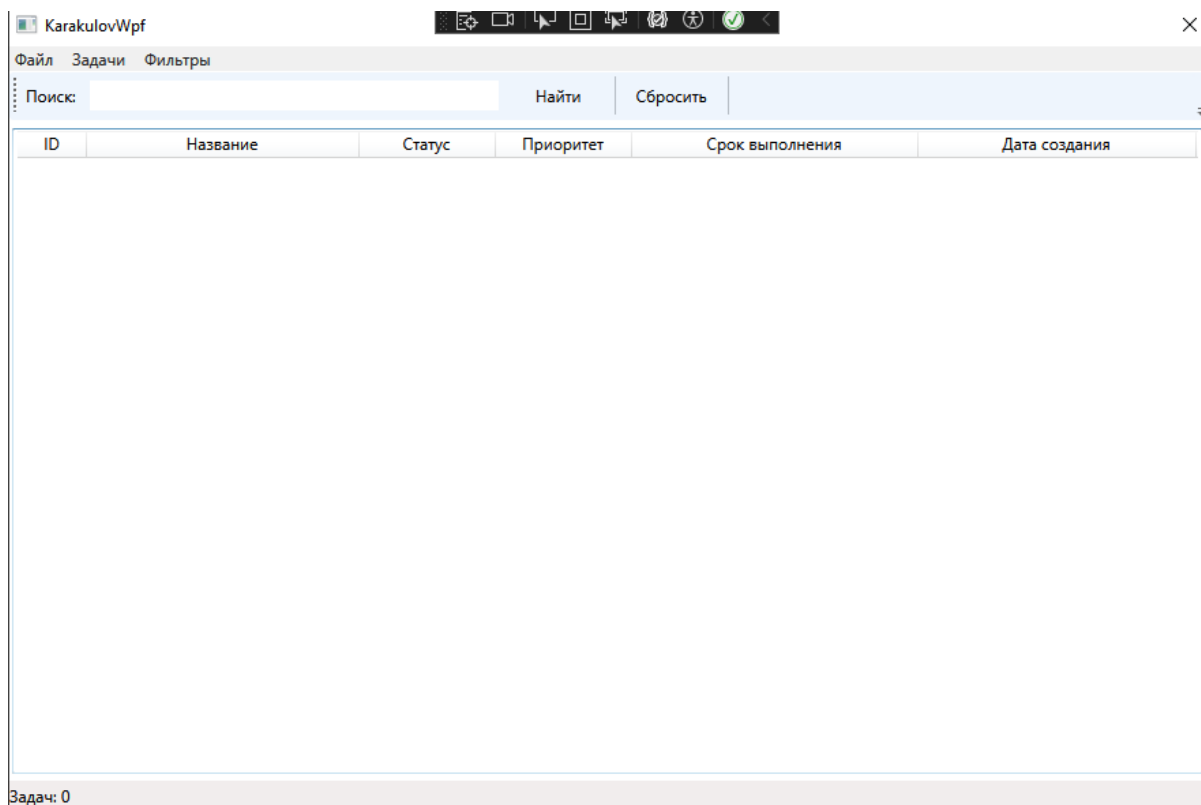
Технические особенности:

- 1) Разметка: Grid (7 строк × 2 колонки)
- 2) Визуальное оформление: отступы между элементами
- 3) Кнопки управления размещены в StackPanel справа
- 4) DataBinding для связи полей со свойствами задачи
- 5) Выпадающие списки заполняются при инициализации
- 6) Обработчики событий Click для кнопок

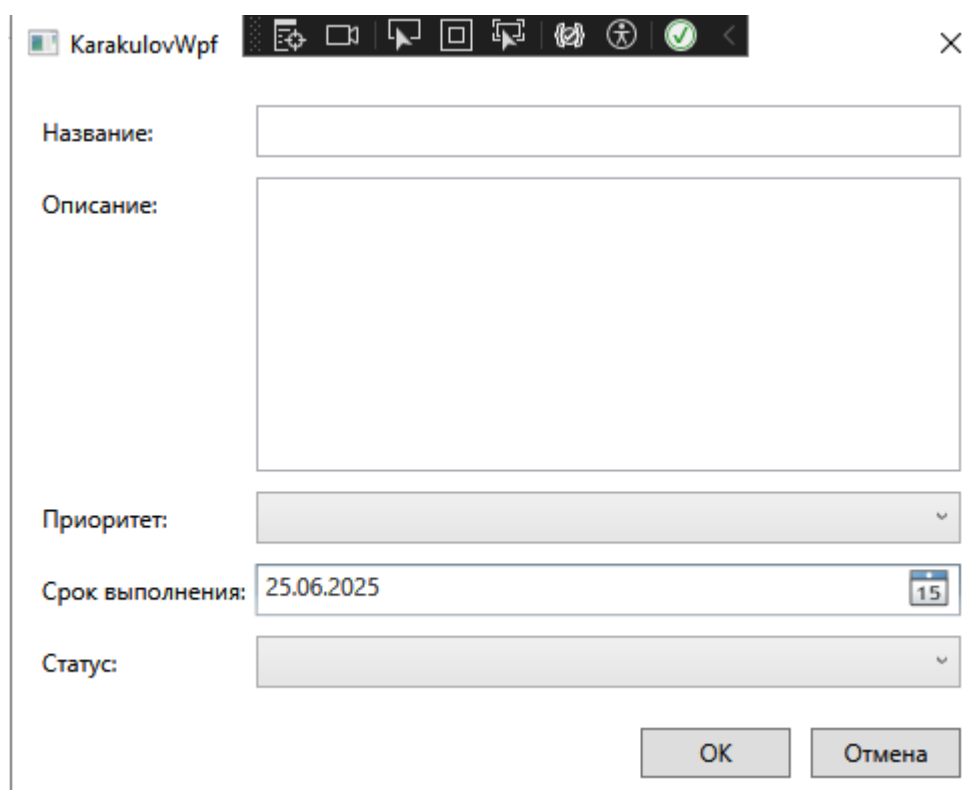
Логика работы:

1. При открытии:
 - 1) При редактировании - поля заполняются текущими значениями
 - 2) При создании - поля пустые
2. При нажатии "ОК":
 - 1) Проверка данных
 - 2) Сохранение в объект задачи
 - 3) Закрытие с DialogResult.OK
3. При нажатии "Отмена":
 - 1) Отмена изменений
 - 2) Закрытие с DialogResult.Cancel

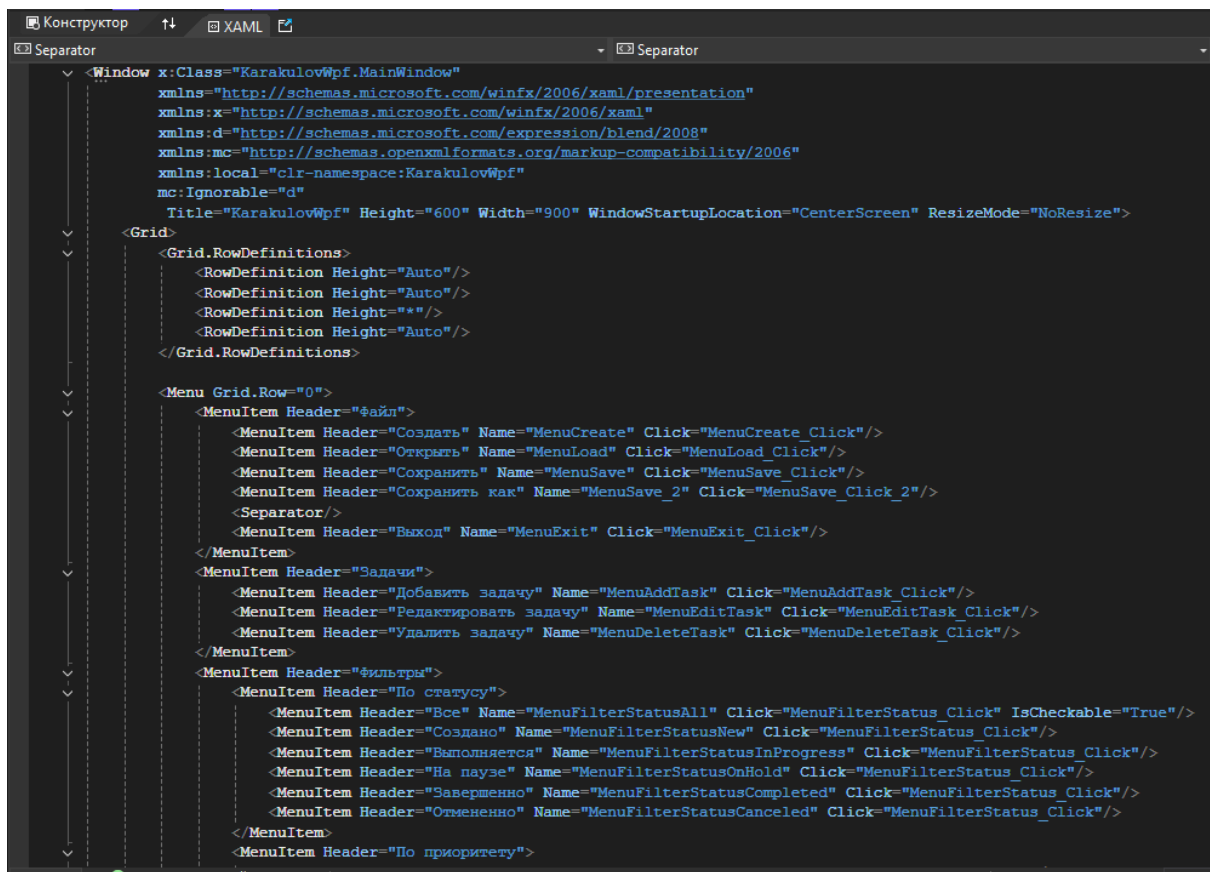
После создания интерфейса для программы, видим результат (См. Рис. 5, 6, 7.1, 7.2, 7.3, 8)



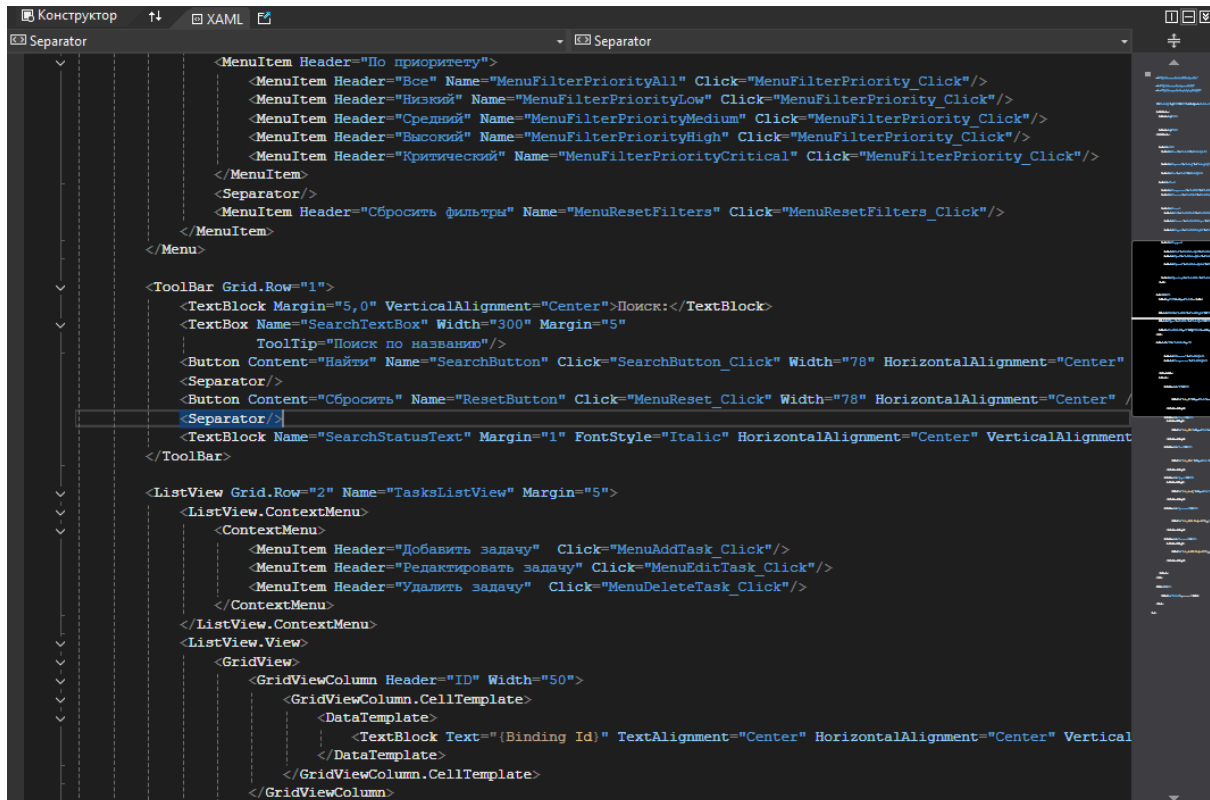
(Рис. 5 – Основное окно MainWindow)



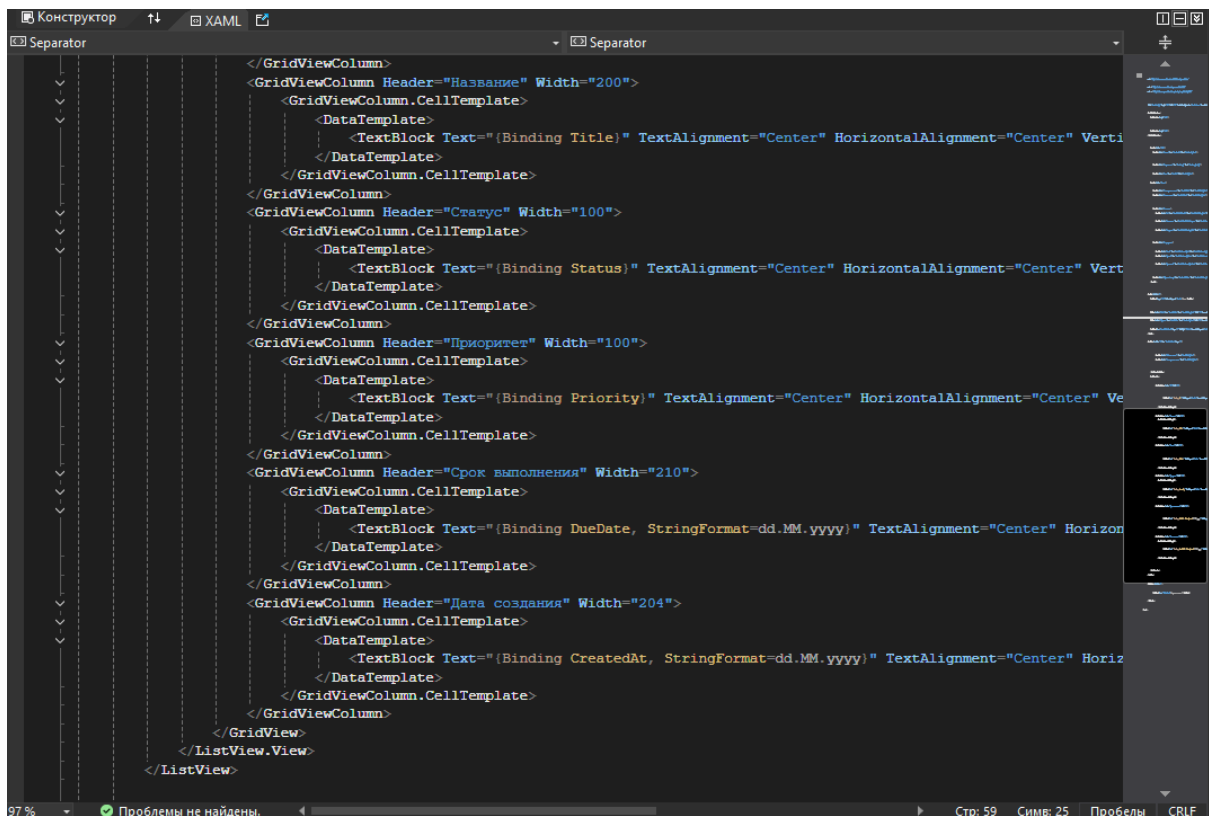
(Рис.6 – Окно CommandWindow)



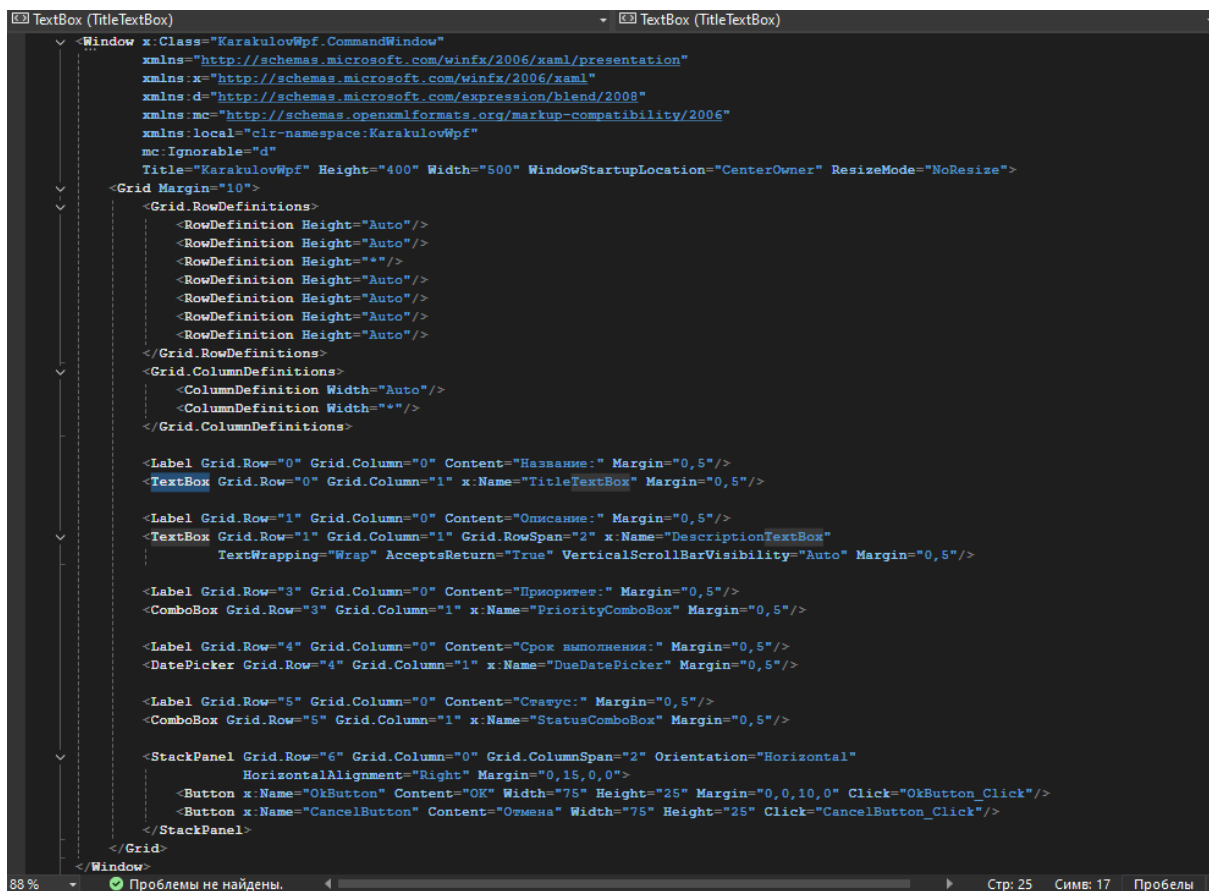
(Рис.7.1 – окно MainWindow)



(Рис.7.2 – окно MainWindow)



(Рис. 7.3 – окно MainWindow)



(Рис. 8 – окно CommandWindow)

Описание реализации функционала окна MainWindow.xaml.cs

При старте приложения выполняется следующая логика:

- 1) Создается экземпляр `_taskManager`, управляющий задачами.
- 2) Настраивается представление списка задач (`TasksListView`) через привязку к источнику данных (`CollectionViewSource`).
- 3) Устанавливаются исходные настройки фильтров: изначально выбираются все возможные статусы и приоритеты.

При закрытии окна освобождаются ресурсы посредством метода `_taskManager.Dispose()`.

Для фильтрации и поиска выполняются такие действия:

- 1) Поисковая строка передает запрос на фильтрацию задач.
- 2) Фильтры по статусу и приоритету задают условия отбора задач и применяются автоматически при изменении значений фильтров.

Операции над задачами включают:

- 1) Добавление новой задачи: Пользователь вводит данные в окне `CommandWindow`. Затем новая задача добавляется методом `_taskManager.AddTask`.
- 2) Редактирование существующей задачи: Выбранная задача передается в окно `CommandWindow` для редактирования, после чего изменяется через `_taskManager.UpdateTask`.
- 3) Удаление задачи: Подтверждение удаления осуществляется окном предупреждения, затем задача удаляется методом `_taskManager.RemoveTask`.

Работа с файлами организована следующим образом:

- 1) Создание нового файла: Диалог сохранения предлагает выбрать путь для нового файла, затем используется метод `_taskManager.CreateFile`.

2) Загрузка файла: Через диалог открытия файла вызывается метод `_taskManager.LoadFromFile`, загружающий ранее сохраненные задачи.

3) Сохранение текущего набора задач: Используется метод `_taskManager.SaveToFile`, который записывает изменения в существующий файл. Если файл не указан, выдается уведомление об ошибке.

4) Операция "Сохранить как": Открывается диалог выбора пути, после чего задачи сохраняются новым файлом.

Также предусмотрены дополнительные возможности:

- 1) Возможность сброса всех примененных фильтров и очистки строки поиска для полного восстановления первоначального представления задач.
- 2) Обновление статусной строки интерфейса, показывающей общее число задач и дополнительную информацию о выполненных операциях (например, "Файл успешно сохранён").

Описание функциональности окна `CommandWindow.xaml.cs`

Это окно предназначено для добавления новых задач либо редактирования существующих.

Основные шаги работы окна:

- 1) **Инициализация:** Базовый конструктор создаёт новую задачу, второй — принимает задачу для редактирования и копирует её во временный объект, предотвращая прямое изменение исходника.
- 2) **Элементы интерфейса:** Выпадающие списки приоритета и статуса наполняются стандартными значениями («Критический», «Высокий», «Средний», «Низкий») и

(«Создано», «Выполняется», «На паузе», «Завершено», «Отменено»), соответственно. Поле срока выполнения устанавливается текущей датой.

Обработка кнопок:

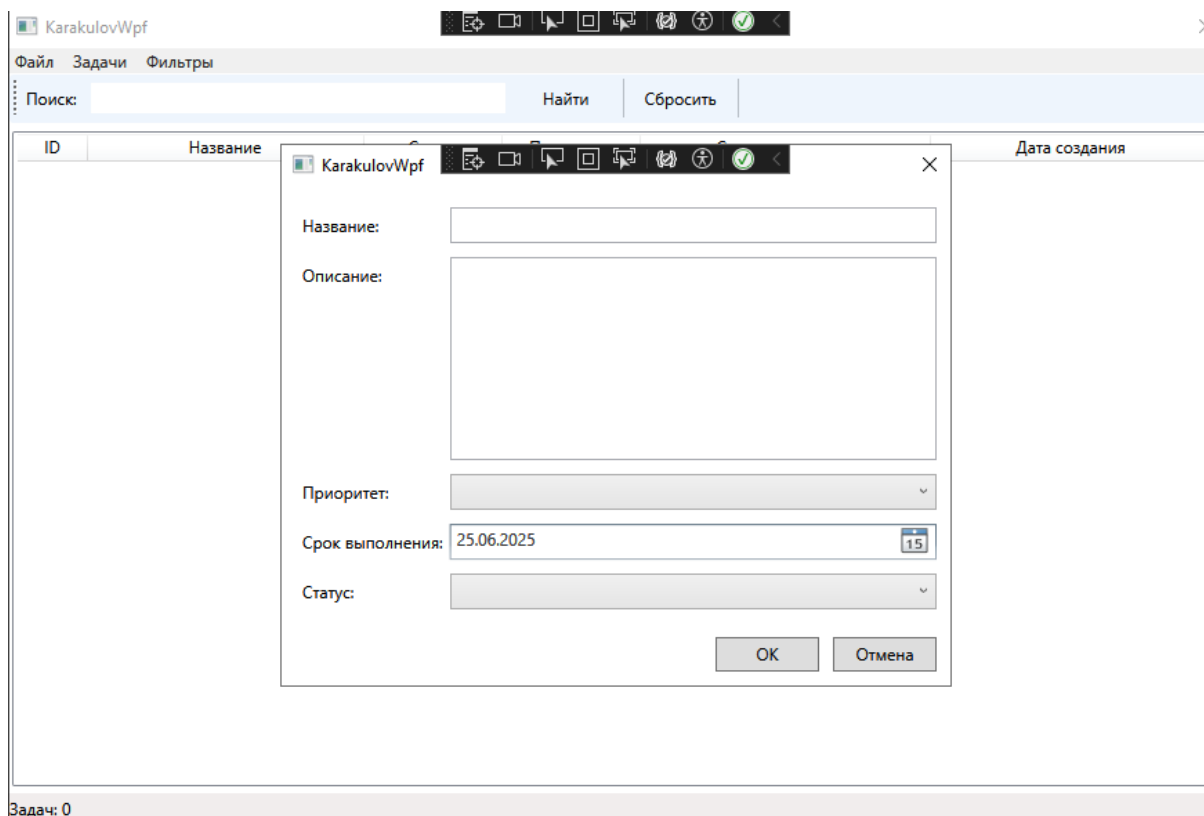
- 1) Кнопка «ОК»: Проверяются введённые данные, включая наличие имени задачи и корректность даты выполнения. Если проверка пройдена, данные сохраняются в задаче, оригинальная задача обновляется, и окно закрывается с результатом True.
- 2) Кнопка «Отмена»: Окно закрывается без сохранения внесённых изменений.

Проверка и сохранение данных: Метод проверки валидности (ValidateInput) контролирует правильность введённого названия и актуальность даты завершения. Данные задачи сохраняются в соответствующую структуру модели через метод SaveTaskData.

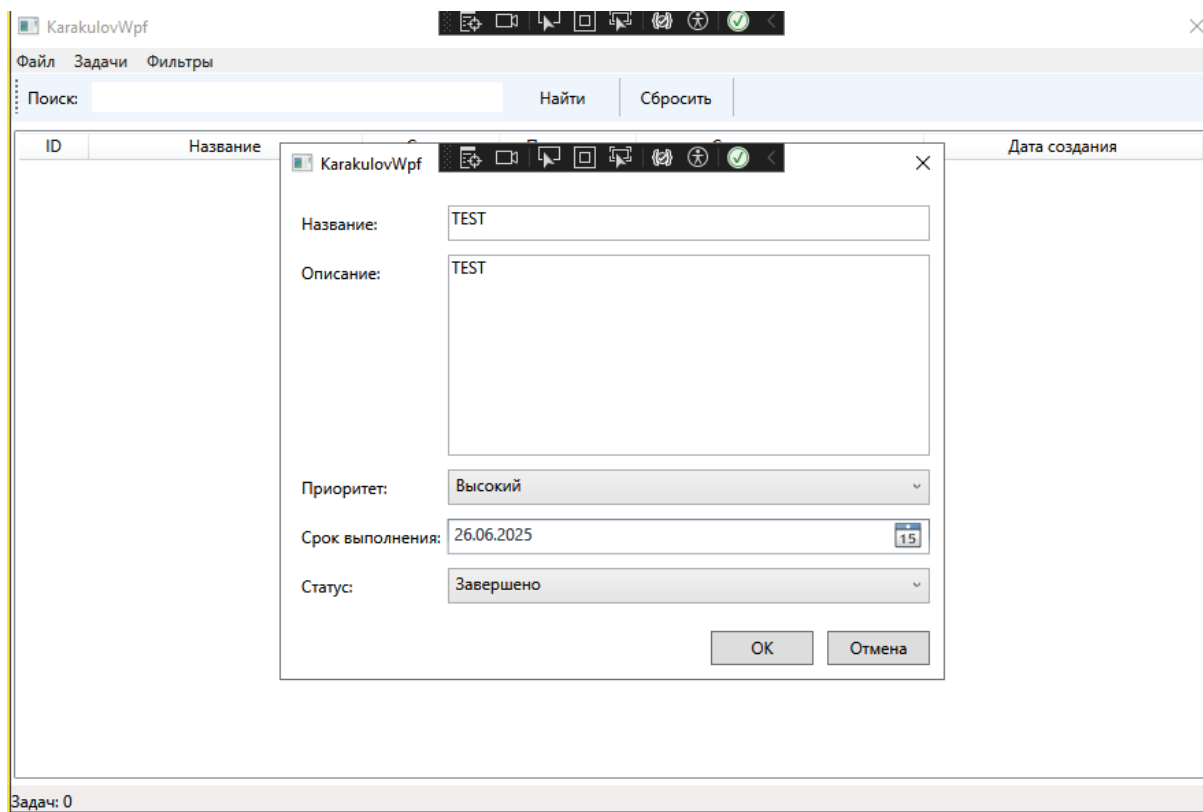
Таким образом, оба окна обеспечивают полный цикл операций по управлению задачами: создание, редактирование, удаление, загрузку/сохранение файлов и работу с интерфейсными элементами для удобства пользователя.

4. Тестирование завершено WPF приложения

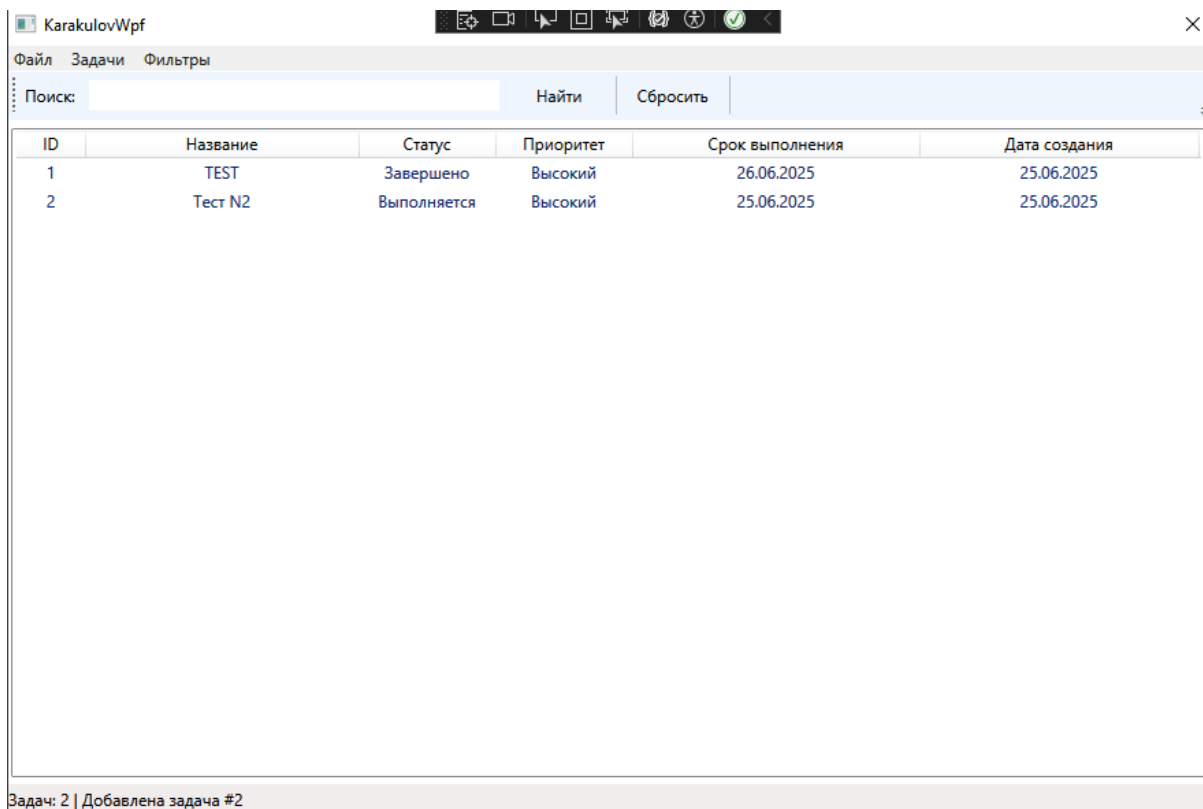
После всех вышеописанных проделанных действий, выполняем запуск готового приложения и пробуем проверить функционал. Выполняем CRUD-операции, применяем различные фильтры и пробуем поработать с файлами «JSON». Результаты и ход действий можно увидеть на рисунках ниже:



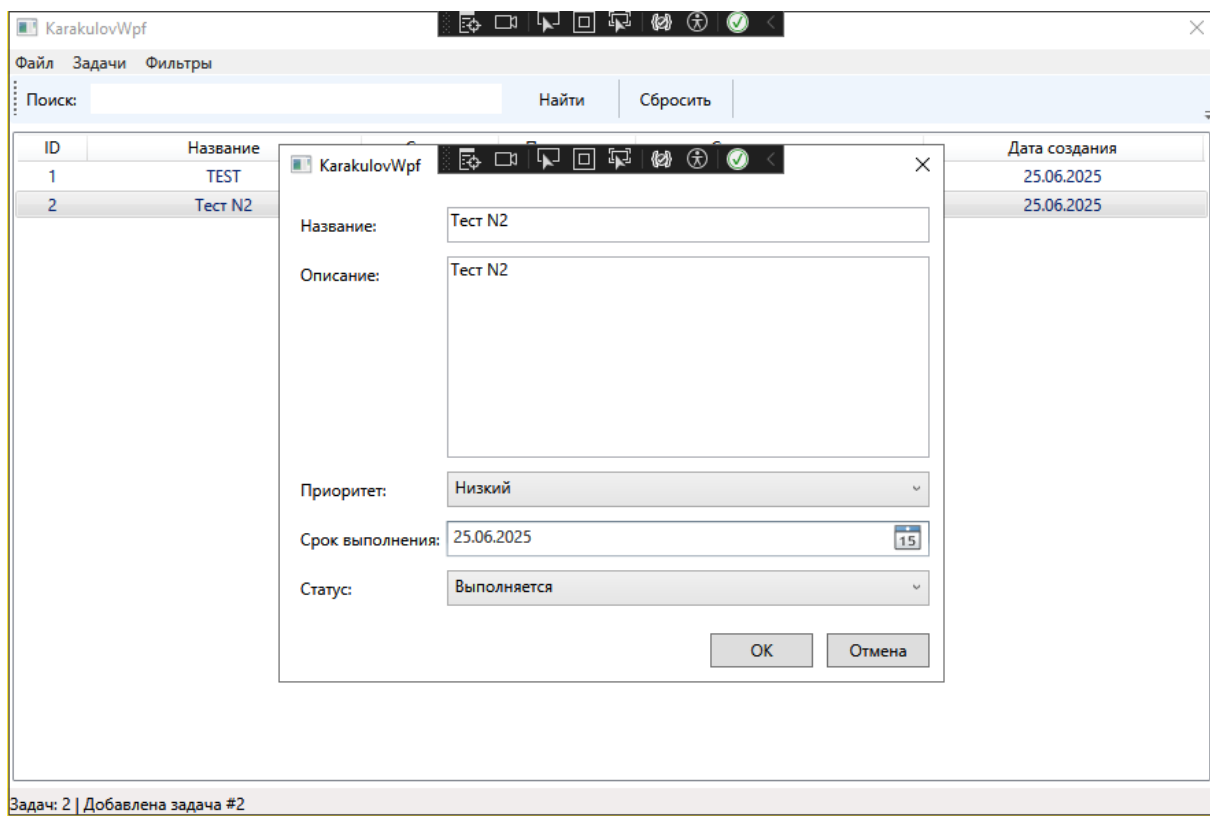
(Рис.9 – Добавление задачи)



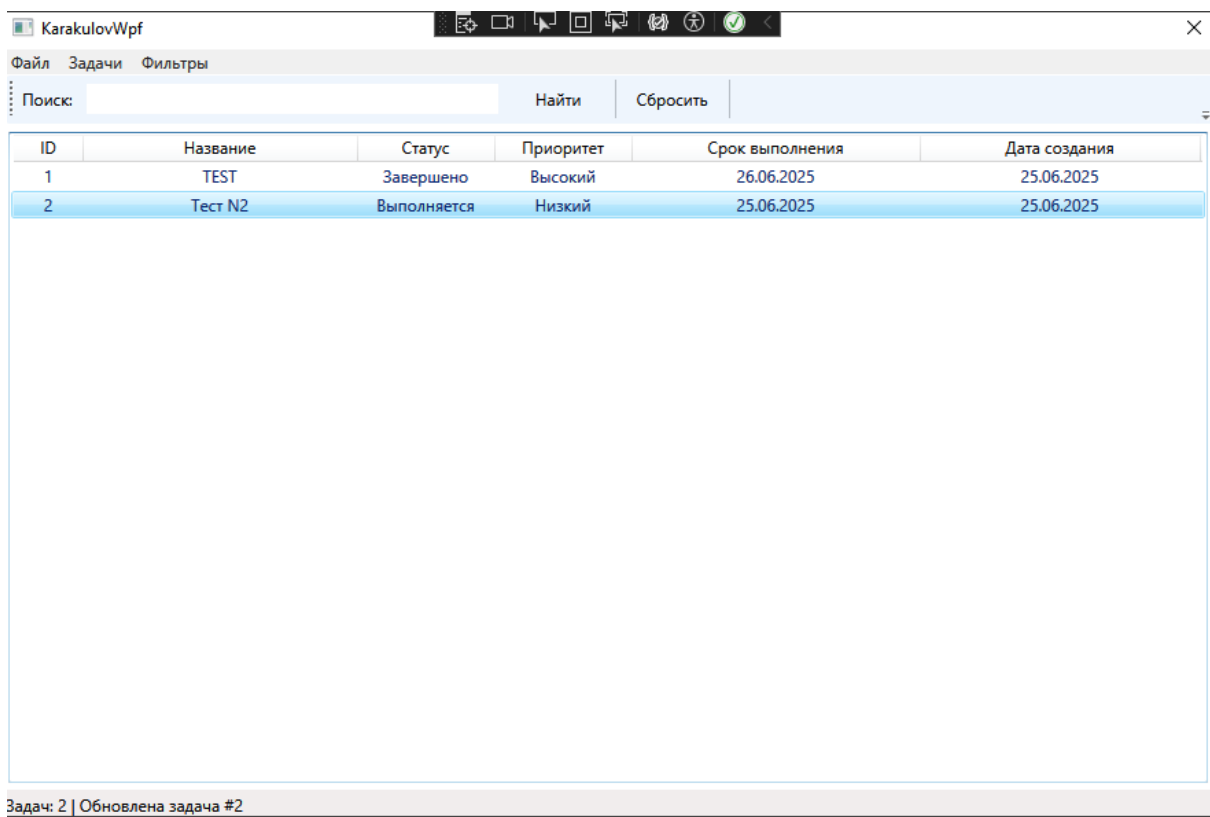
(Рис.10 – Добавление задачи – Описывание «Названия» и «Описания»)



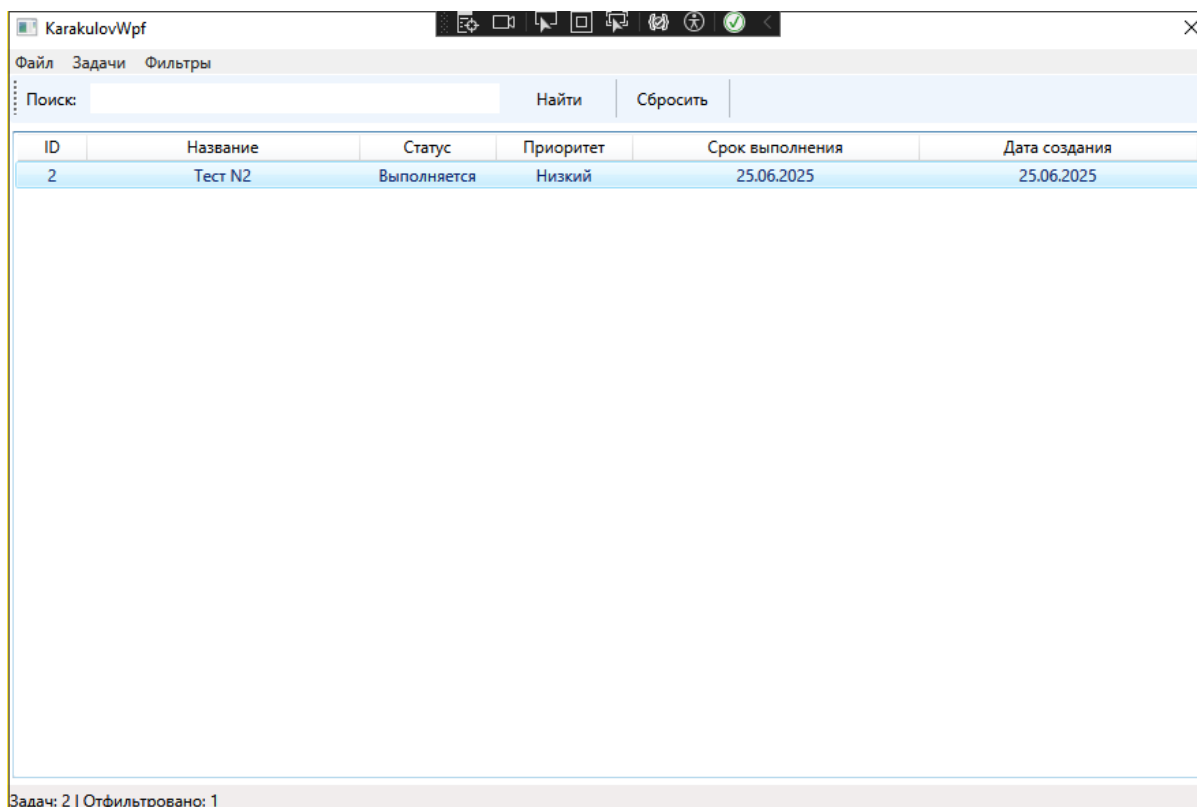
(Рис.11 – Полученный результат при создании и добавлении задач)



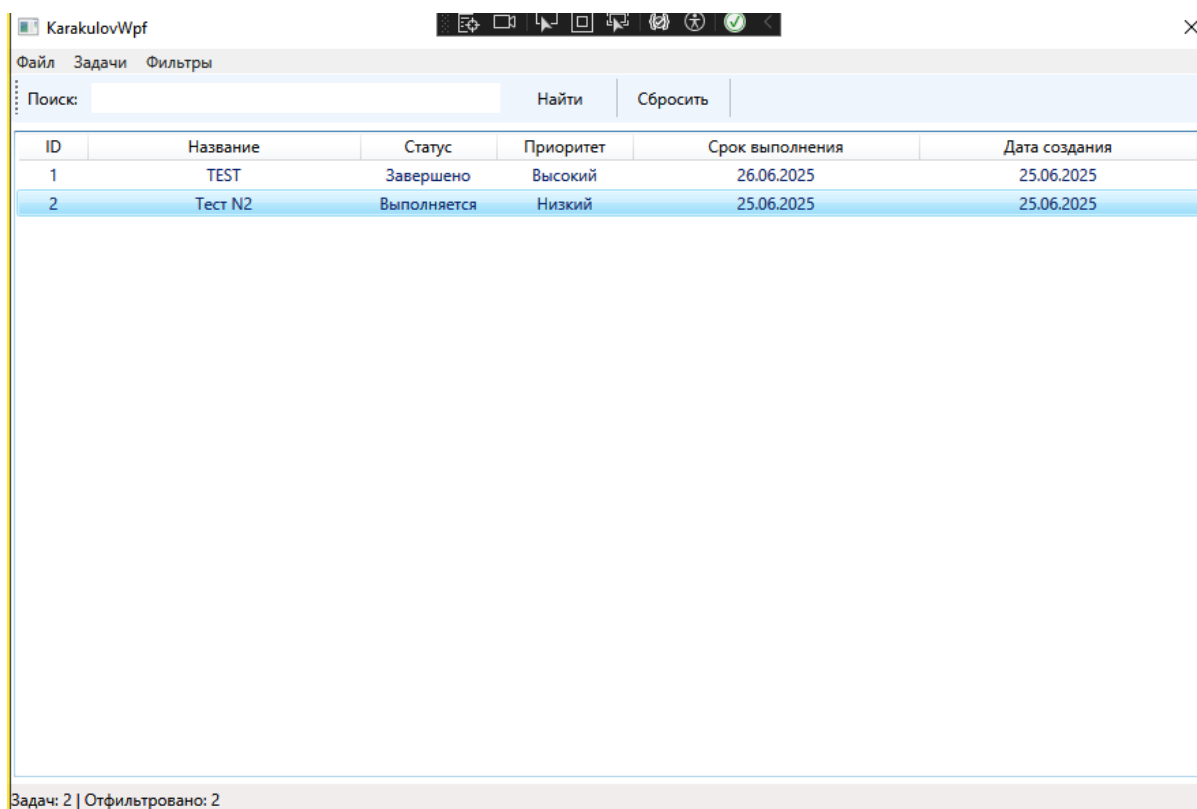
(Рис.12 – Редактирование задач)



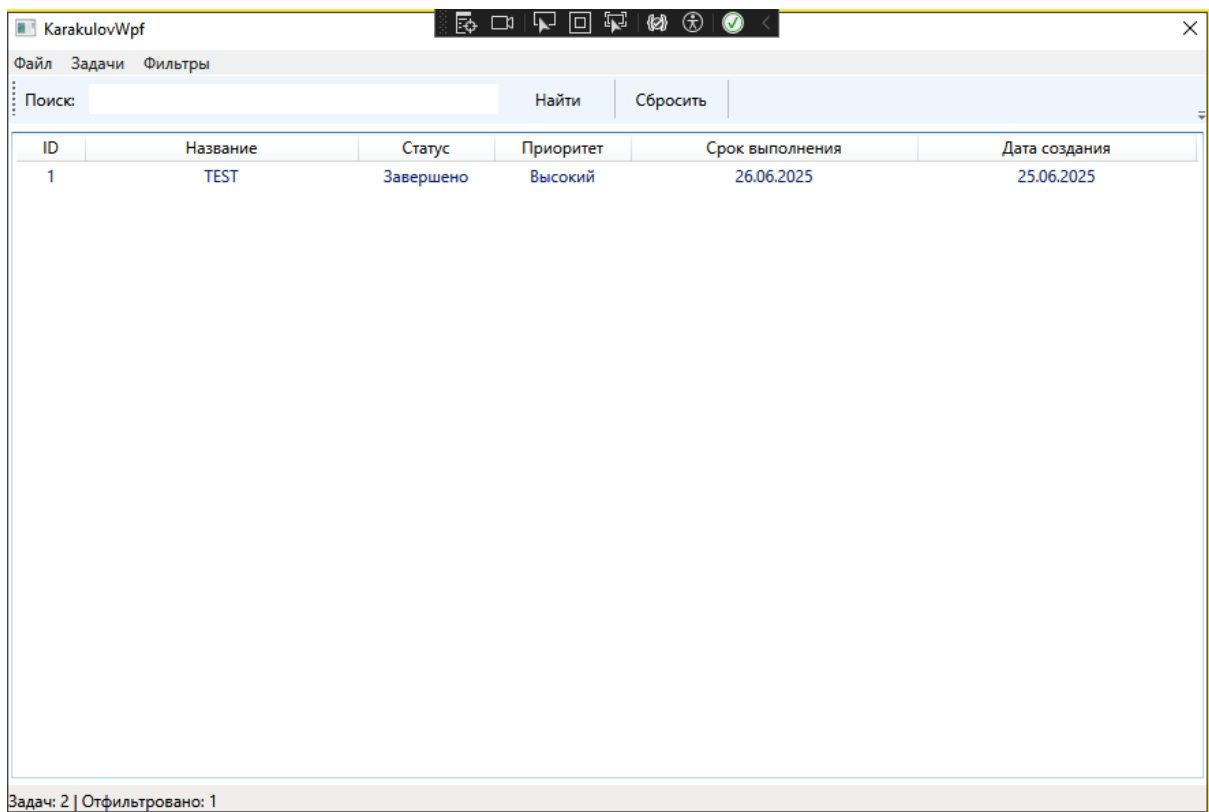
(Рис.13 – Полученный результат)



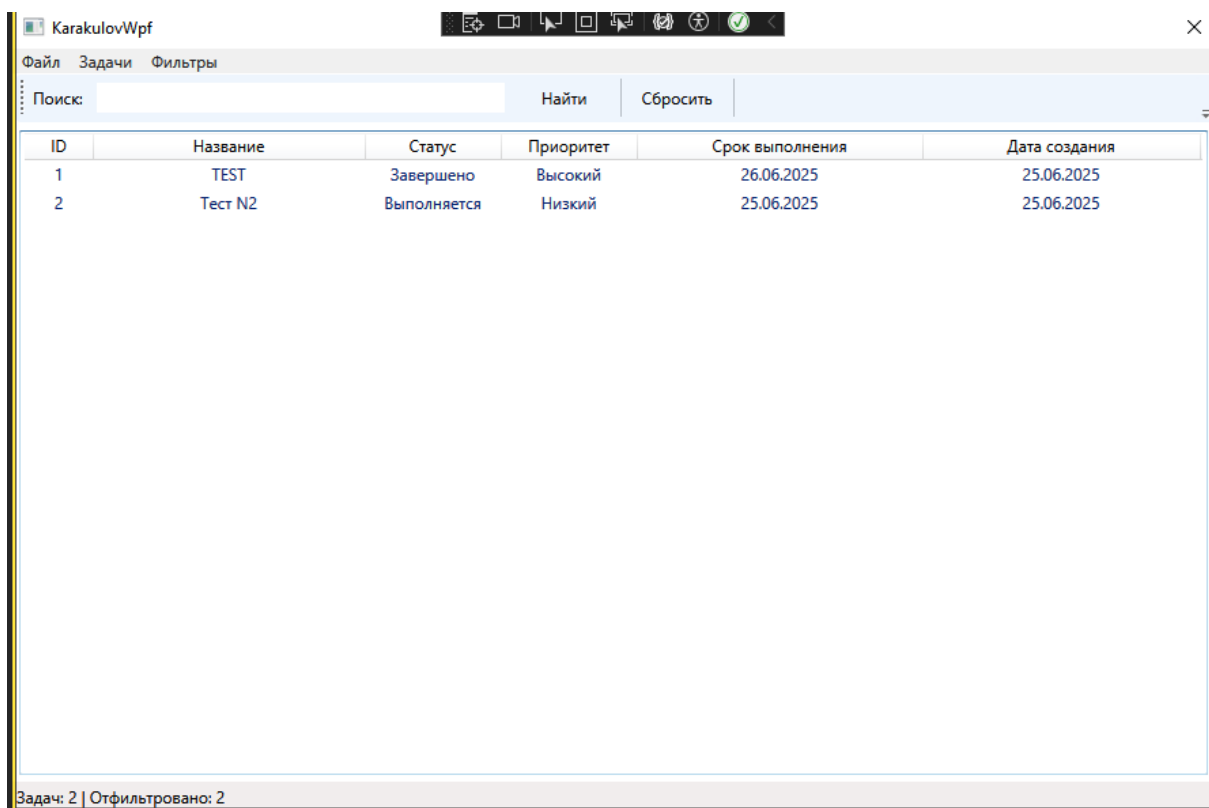
(Рис.14 – Применение фильтра только для тех задач, которые «Выполняются»)



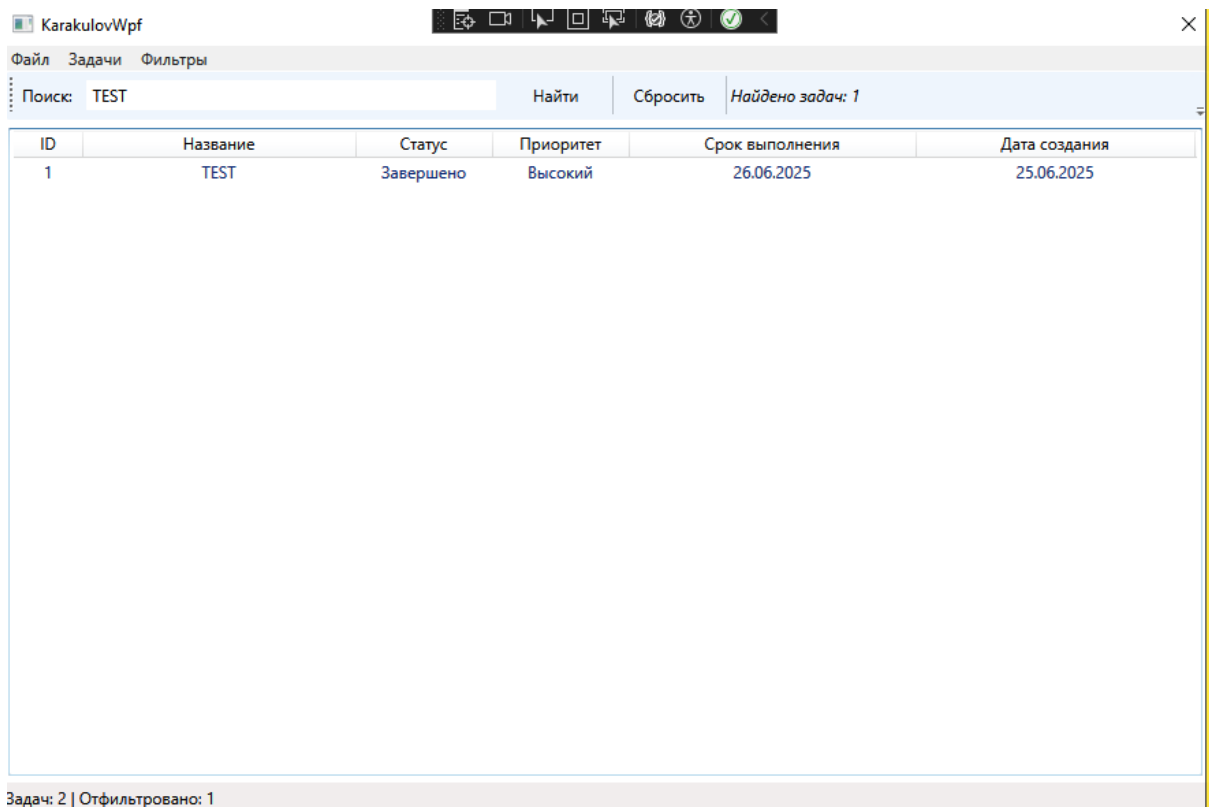
(Рис.15 – Применение фильтра – «Все»)



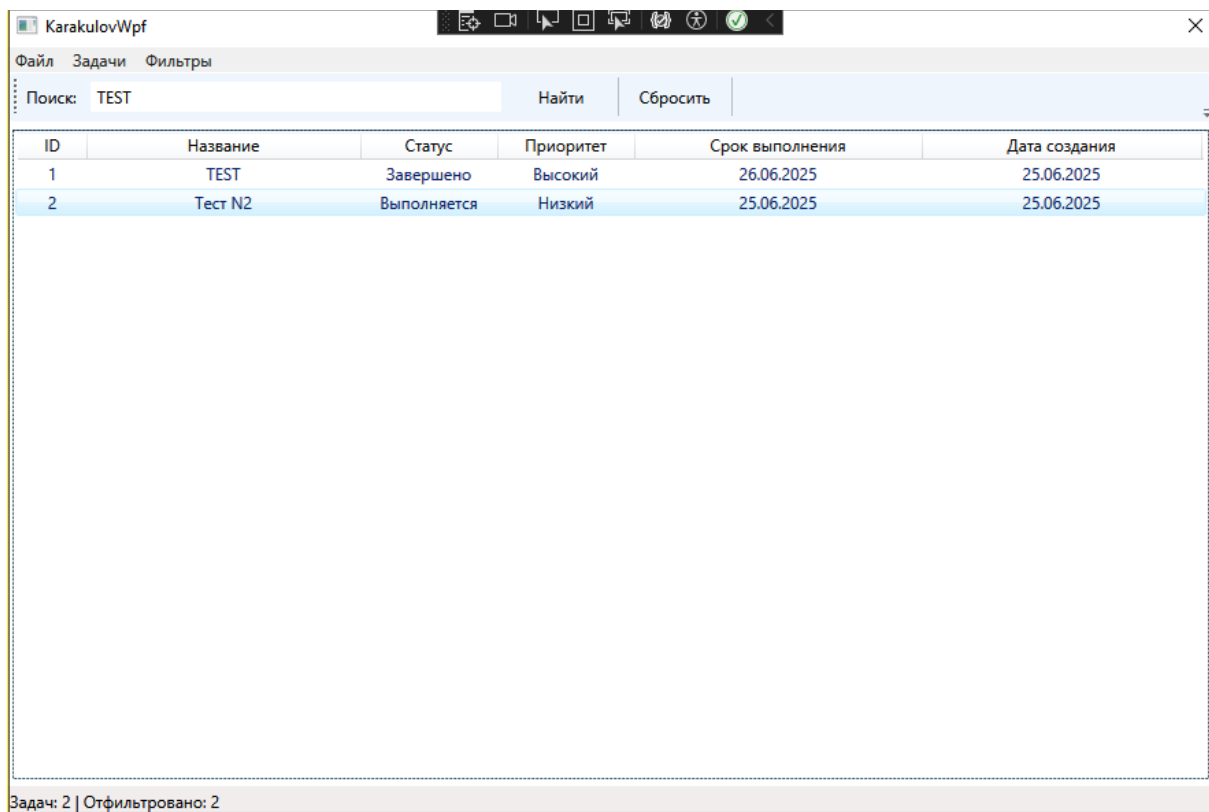
(Рис.16 – Применение фильтра по приоритетности, в данном случае – «Высокий»)



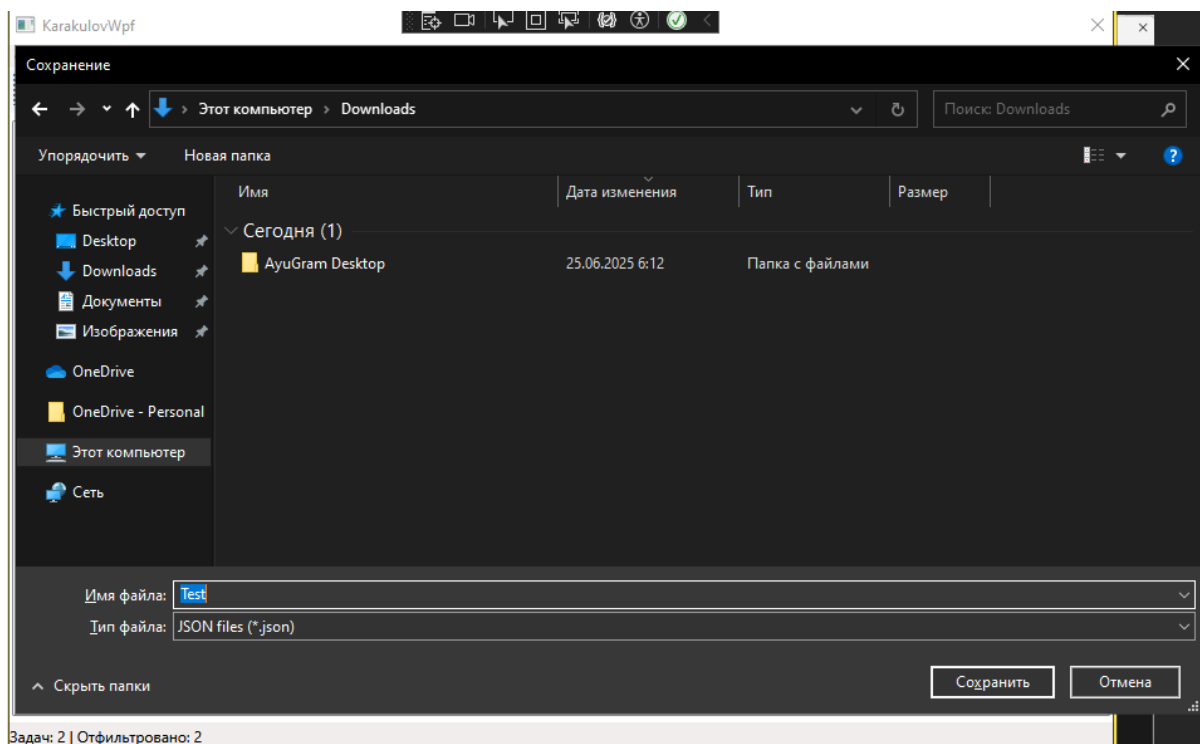
(Рис.17 – Результат сброса всех фильтров)



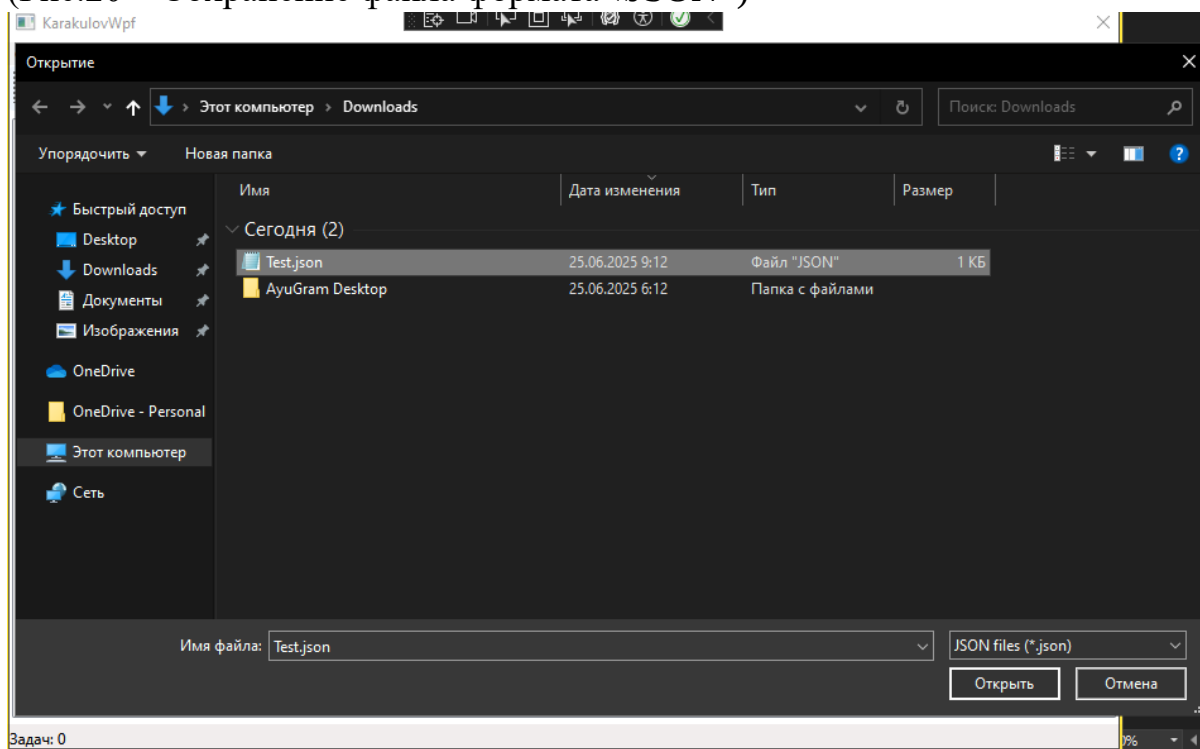
(Рис.18 – Поиск задачи по названию, в данном случае название которое мы ищем из списка «TEST»)



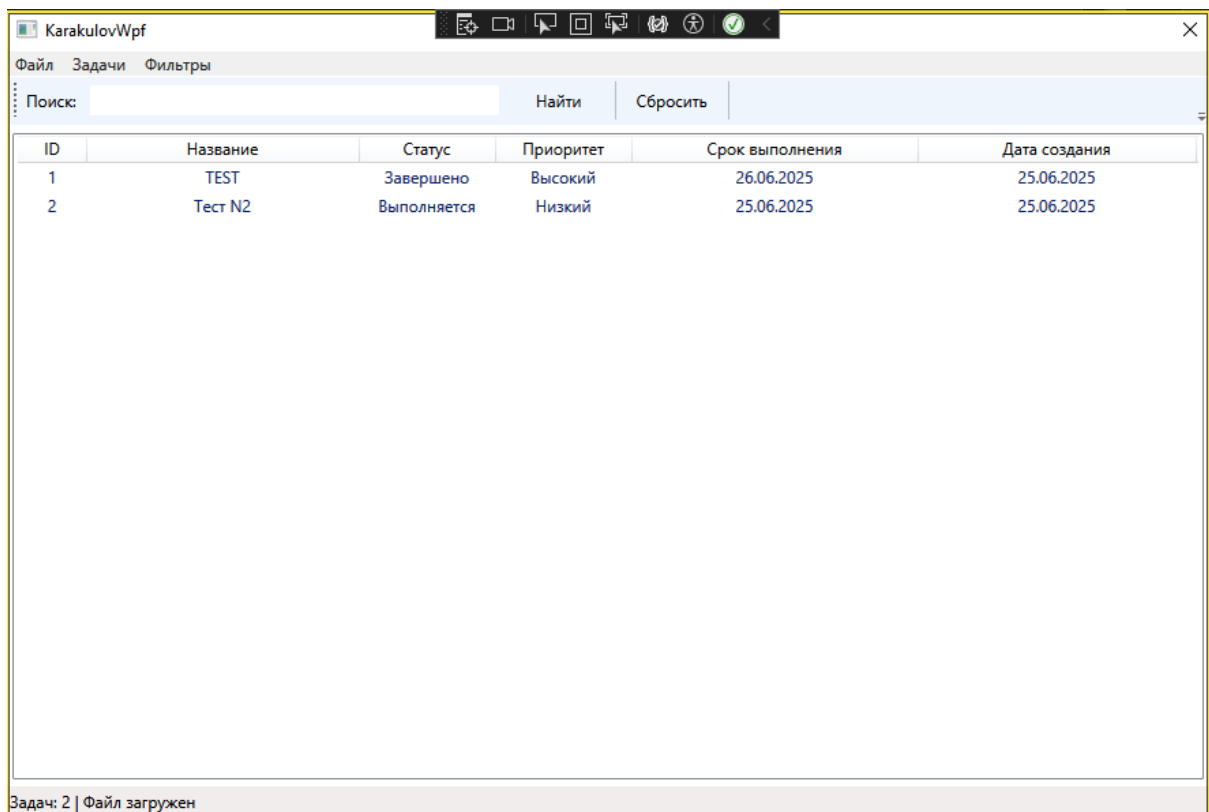
(Рис.19 – Результат сброса поиска по названию при помощи кнопки «Сбросить»)



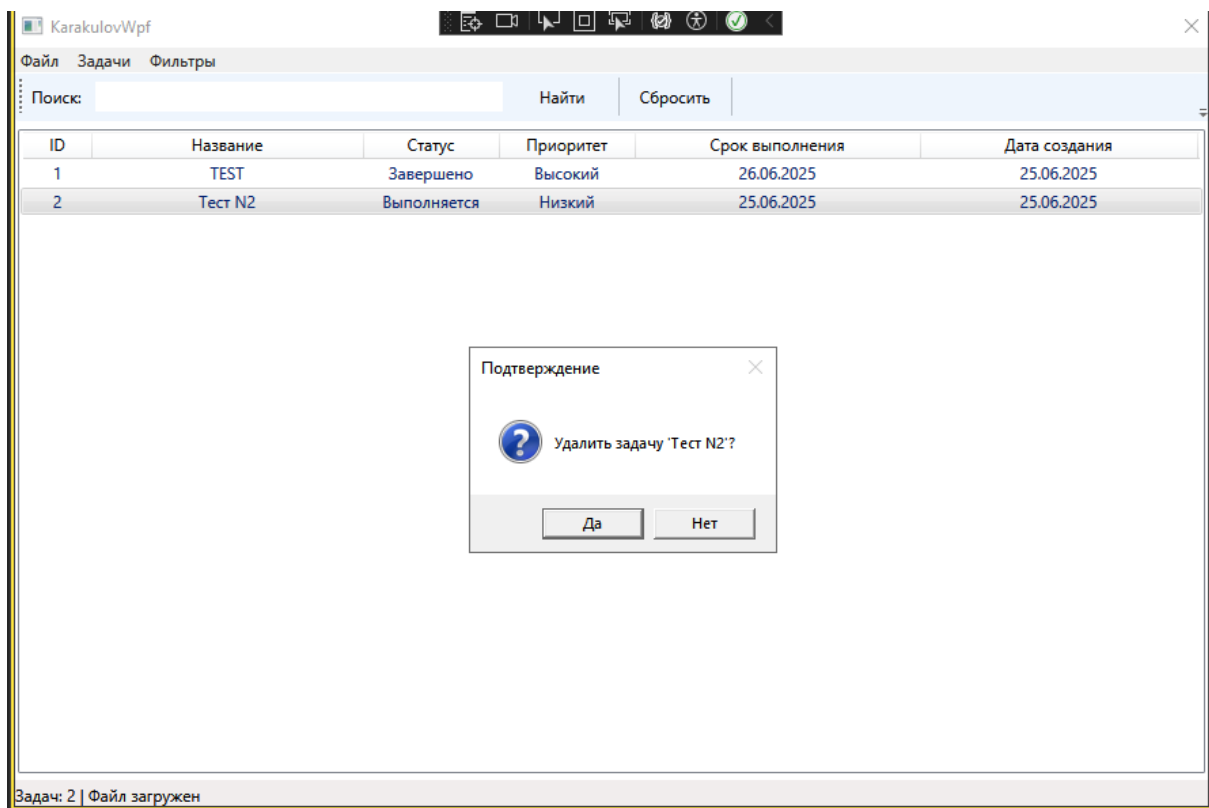
(Рис.20 – Сохранение файла формата «JSON»)



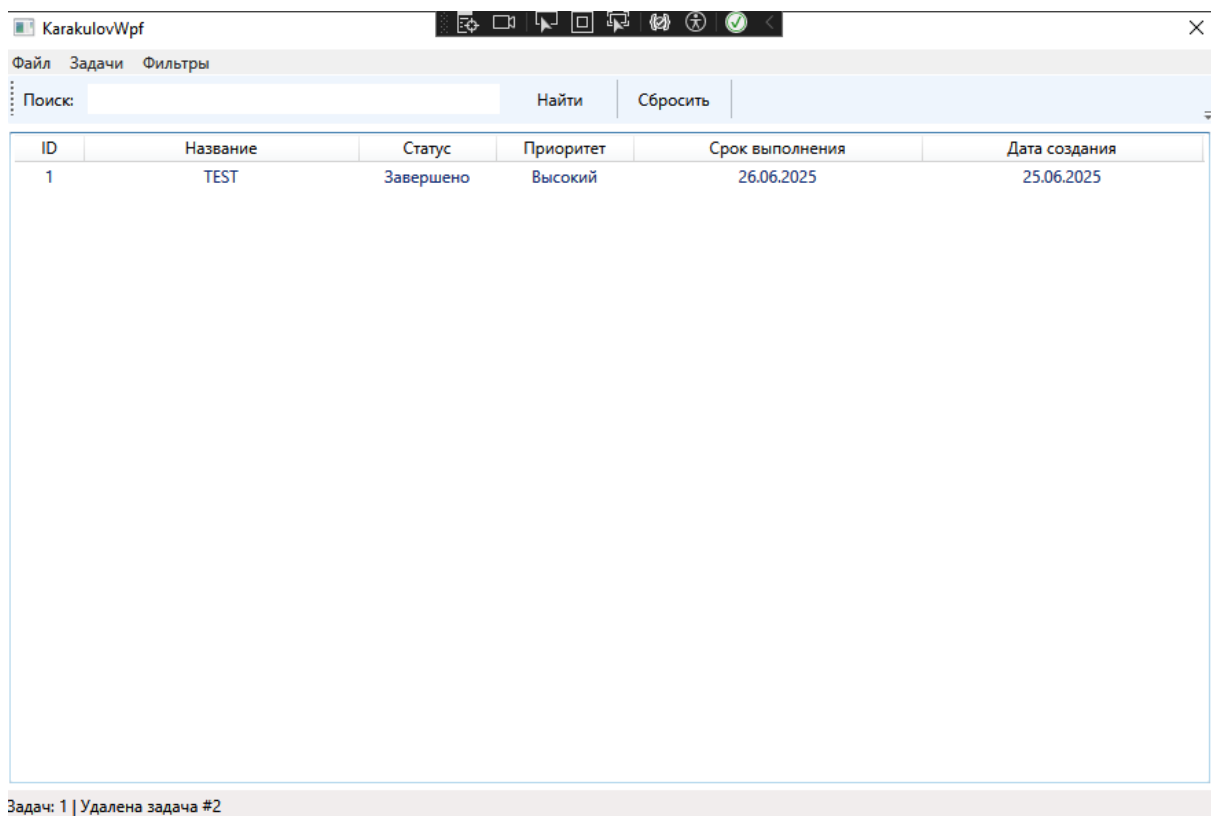
(Рис.21 – Открываем по-новому программу, и добавляем заранее экспортированный файл формата «JSON»)



(Рис.22 – Результат открытия файла «Test.json»)



(Рис.23 – Удаление задачи)



(Рис.24 – Результат удаление выбранной нами задачи)

Вывод

Итоги выполненной работы заключаются в создании эффективного приложения на платформе WPF, позволяющего удобно и эффективно оперировать данными, хранящимися в оперативной памяти в форме коллекции объектов. Ключевая функциональность программы была вынесена в отдельную библиотеку классов .NET Framework, что существенно улучшило структурированность проекта и облегчило процесс тестирования отдельных компонентов. Благодаря такому разделению логики и пользовательского интерфейса стало возможным быстро выявлять и устранять потенциальные проблемы, значительно повысив надежность и качество конечного продукта.

Кроме того, важной частью процесса разработки стала реализация модульных тестов, охватывающих основные операции приложения, такие как добавление, обновление, удаление и выборка данных. Эти тесты помогли убедиться в правильности функционирования ключевых функций и гарантировали стабильность поведения системы даже при внесении изменений в кодовую базу.

Графическая оболочка приложения выполнена с использованием возможностей платформы WPF, предоставляющей гибкость и простоту в проектировании визуальных компонентов. Интерфейс получился удобным и понятным для пользователей, обеспечивая быстрый доступ ко всем необходимым операциям по работе с данными. Данный подход продемонстрировал эффективность современной архитектуры приложений, сочетающей выразительность пользовательского интерфейса и надежную бизнес-логику.

Полученный опыт стал ценным вкладом в развитие профессиональных компетенций, связанных с разработкой десктопных приложений. Были освоены важные аспекты программирования на C# и WPF, такие как работа с коллекциями, построение тестируемых архитектур, организация модулей

и разработка качественных пользовательских интерфейсов. Таким образом, выполнение данной работы представляет собой важный этап подготовки специалиста, готового создавать современные, надежные и удобные решения для реальных прикладных задач.